6. PROGRAMAREA IN LIMBAJ DE ASAMBLARE A CALCULATORULUI DIDACTIC

OBIECTIVE

CUPRINS

6. PROGRAMAREA IN LIMBAJ DE ASAMBLARE A CALCULATORULUI DIDACTIC 6-6

6.1 Generalităţi privind limbajul de asamblare 6-7

6.2 Organizarea spaţiului de memorie 6-9

6.3 Organizarea spaţiului de I/E 6-9

6.4 Formatul instrucţiunilor 6-9

6.5 Moduri de adresare 6-14

6.6 Elementele limbajului de asamblare 6-14
6.6.1 Alfabetul limbajului de asamblare 6-14
6.6.2 Cuvinte 6-15
6.6.2.1 Identificatori 6-15
6.6.2.2 Constantele 6-15
6.6.2.2.1 Constantele întregi 6-15
6.6.2.2.2 Constante reale 6-16
6.6.2.2.3 Constante zecimal codificate binar 6-16
6.6.2.2.4 Constante exprimate prin şiruri 6-16
6.6.3 Propoziţii 6-17
6.6.3.1 Operatori 6-17
6.6.3.1.1 Operatori aritmetici 6-17
6.6.3.1.2 Operatori de deplasare 6-18
6.6.3.1.3 Operatori logici 6-18
6.6.3.1.4 Operatori relaţionali 6-18
6.6.3.1.5 Operatorul DUP 6-18
6.6.3.2 Prioritatea operatorilor 6-19

6.7 Declararea datelor în limbajul de asamblare 6-19

6.8 Pseudoinstrucţiuni 6-20
6.8.1 Pseudoinstrucţiunea ORG 6-20
6.8.2 Pseudoinstrucţiunea EQU 6-21
6.8.3 Pseudoinstrucţiunea = 6-21
6.8.4 Pseudoinstrucţiunea DW 6-21
6.8.5 Pseudoinstrucţiunea DD 6-21
6.8.6 Pseudoinstrucţiunea DQ 6-21

6.1 Generalităţi privind limbajul de asamblare

Limbajele de programare sunt rezultatul unei evoluţii, determinată în principal de evoluţia structurii şi arhitecturii calculatoarelor şi a domeniilor de aplicaţii.

Fiecare calculator dispune de un limbaj propriu, numit limbaj maşină. Limbajul maşină utilizează un alfabet binar ({0,1}) pentru reprezentarea instrucţiunilor maşină. Fiecare instrucţiune maşină este reprezentată de o secvenţă de 0 şi 1 conform unor reguli specifice de formare a instrucţiunilor în limbajul respectiv. Un program într-un asemenea limbaj maşină poate fi reprezentat ca o secvenţă de 0 şi 1.

Instrucţiunile maşină ale limbajului descriu operaţii aritmetice şi logice, operaţii de transfer date între diversele resurse ale maşinii de bază, operaţii de transfer control, etc.

Limbajul maşină al calculatorului didactic va fi definit acum, după faza de elaborare a unităţii de comandă a acestuia. De altfel, instrucţiunile maşină sunt citite, interpretate şi executate de către unitatea de comandă a calculatorului didactic aşa cum s-a văzut în capitolul precedent.

Programul în limbaj maşină poartă numele de *"PROGRAM OBIECT"* şi reprezintă forma executabilă a acestuia. Elaborarea unui program în limbaj maşină este o operaţie extrem de dificilă, iar în cazul unor arhitecturi de calculatoare de tip CISC (Complex Instruction Set Computer) practic imposibilă.

Pentru a permite elaborarea de programe care să utilizeze resursele calculatorului, foarte aproape de posibilităţile oferite de limbajele maşină şi, în acelaşi timp, să elimine o bună parte din neajunsurile acestuia, s-a creat limbajul de asamblare.

Fiecărei instrucţiuni maşină i se asociază o instrucţiune în limbaj de asamblare (dar nu întotdeauna corespondenţa este de 1 la 1), numai că, în loc de codificarea binară a informaţiilor cuprinse într-o instrucţiune maşină, se foloseşte o codificare simbolică.

Limbajul de asamblare se caracterizează prin aceea că:

- principalele sale operaţii sunt operaţiile de bază ale calculatorului;
- codurile operaţiilor se specifică prin mnemonice asociate;
- localizarea instrucţiunilor şi a operanzilor se face simbolic.

Limbajul de asamblare are o serie de avantaje, dar şi dezavantaje.

Dintre avantaje amintim:

- posibilitatea de control foarte intim a resurselor calculatorului;
- eficienţă maximă în implementarea unor funcţii;

Dintre dezavantaje amintim:

- instrucţiunile specifică operaţii elementare, ceea ce conduce la un efort mare în elaborarea programelor complexe;
- nu este adecvat pentru implementarea prelucrărilor matematice;
- fiind specific unui tip de calculator, programele nu sunt portabile;
- elaborarea programelor în limbaj de asamblare necesită cunoştinte despre structura şi arhitectura calculatorului;
- productivitatea unui programator exprimată în număr de linii de program sursă este practic aceeaşi indiferent de limbajul de programare utilizat. În mod corespunzător, efortul de scriere pentru o aceaşi aplicaţie este mult mai mare în cazul în care se utilizează limbajul de asamblare.

Programele sursă elaborate în limbaj de asamblare trebuie să fie translatate în programe obiect, direct executabile de către maşina de bază. Procesul de translatare se realizează cu ajutorul unui program ASAMBLOR, cum se vede în Fig. 6.1.



Figura 6.1 Asamblarea unui program.

În general atunci când se elaborează un program, adresa la care acest program va fi încărcat în memorie pentru execuţie nu este fixată. Din acest motiv referirile la adresele de memorie se fac într-o formă simbolică. Dacă programul este elaborat astfel încât poate să fie încărcat pentru execuţie la orice adresă de memorie se spune că programul este relocabil.

În acest caz, programul se scrie ca şi cum ar fi plasat în memorie la adresa zero. Valoarea unei adresei calculate în procesul de asamblare a programului se numeşte deplasament, iar adresa fizică se calculează adunând la deplasament adresa de încărcare în memorie a programului (sau a modulului).

Un asamblor este, de regulă, confruntat cu trei feluri de informaţii simbolice existente în textul programului:

- elemente ***absolute*** a căror valoare nu depinde de modul în care secţiunea care le conţine este plasată în memoria calculatorului. Ca elemente absolute putem enumera: codurile de operaţii specificate prin mnemonice şi constantele.
- elemente ***relocabile*** a căror valoare depinde de poziţia lor în secţiune şi poate diferi de la o execuţie la alta în funcţie de locul în care secţiunea de program se încarcă în memorie. Elementele relocabile sunt de obicei numele simbolice ale celulelor din memorie ce reprezintă instrucţiuni sau operanzi din module relocabile. Adresa fizică a elementelor relocabile se calculează prin adunarea adresei de început a modulului cu valoarea simbolului respectiv fată de începutul modulului.
- elemente de ***referintă externă*** referite într-un modul şi definite în altul şi a căror valoare se stabileşte în momentul legării modulelor într-un program unic.

Rezultatul asamblării este un text sub formă binară în care referinţele externe sunt păstrate sub forma simbolică iniţială în tabele asociate secţiunilor de program. Codul binar al modulelor este însoţit de informaţii ce indică locul referinţelor relocabile, pentru ca la momentul încărcării în memorie valorile acestora să se poată translata în referinţe absolute.

Combinarea diverselor module (secţiuni de program) într-un program executabil se face prin rezolvarea referinţelor externe (înlocuirea numelor simbolice cu adrese de memorie) şi adăugarea, eventual, a unor module din biblioteci standard, şi ele păstrate în forma relocabilă. Aceste operaţii sunt de obicei efectuate de către un editor de legături.

Programul sursă se elaborează utilizând un editor de texte. Prin convenţie, modulele de program sursă au extensia ASM sau MAC, dar aceasta nu este întotdeauna obligatoriu deoarece programele care operează asupra unor fişiere de obicei testează dacă tipul informaţiilor din acel fişier corespunde cu tipul prelucrării.

Modulele de program sursă pot fi organizate astfel încât toate procedurile unui program să fie incluse într-un singur modul, sau să fie repartizate în diverse module.

Pentru asamblarea fiecărui modul al unui program se utilizează asamblorul. Într-un modul se pot include fişiere sursă, pregătite în prealabil pe disc prin specificarea explicită a acestui lucru în modulul care se translatează.

Pentru fiecare modul sursă (fişier .ASM) se creează câte un fişier obiect (fişier .OBJ) şi opţional, un fişier listing (fişier .LST) sau un fişier de referiri încrucişate (fişier .CRF).

Dacă modulul elaborat în limbaj de asamblare va fi legat cu module în limbaje de nivel înalt, atunci acestea trebuie să fie compilate în momentul respectiv (dacă nu au fost compilate în prealabil). Denumirea fişierelor generate de asamblor şi formatul acestora, modul de legare într-un program unitar a modulelor elaborate în diferite limbaje depind de versiuni, firma producătoare şi din păcate nu sunt standardizate.

6.2 Organizarea spaţiului de memorie

Spaţiul de memorie de dimensiuni 64k cuvinte de 16 biţi este omogen, în sensul că toate locaţiile de memorie sunt adresate în mod echivalent. Operaţiile de transfer cu memoria sunt pe 16 biţi. Primele 16 cuvinte sunt rezervate pentru sistemul de întreruperi. La recunoaşterea unei întreruperi se generează un salt indirect prin aceste celule capcană la rutina de tratare corespunzătoare.

6.3 Organizarea spaţiului de I/E

Calculatorul didactic poate adresa direct 256 de porturi de I/E de 16 biţi. Desigur că datele transferate cu echipamentele de I/E pot fi organizate pe 8 sau 16 biţi. Aşa cum s-a specificat în capitolul 5, spaţiul de I/E este separat şi adresarea lui se face prin instrucţiuni dedicate. Avantajele şi dezavantajele acestei soluţii au fost prezentate în capitolul 5.

6.4 Formatul instrucţiunilor

Instrucţiunile calculatorului didactic au lungime variabilă între 1 şi 6 octeţi. Formatul general al instrucţiunilor este prezentat în fig 6.2. Deoarece pentru instrucţiunile de prelucrare de date care se execută asupra a doi operanzi, rezultatul înlocuieşte primul operand, vom numi acest operand operand destinaţie în timp ce pe cel de-al doilea operand îl vom numi operand sursă.

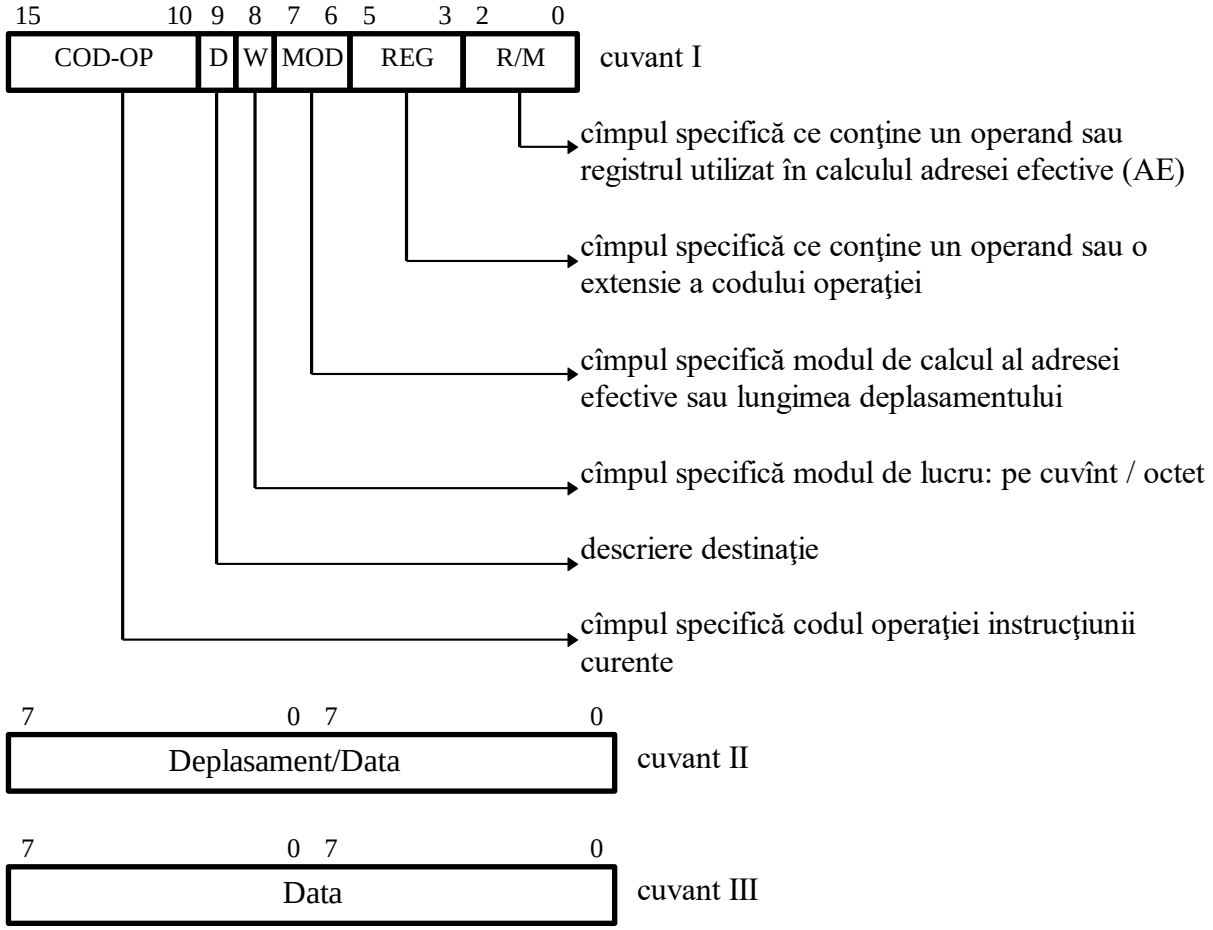


Figura 6.2 Formatul general al instrucţiunilor maşină

unde:

COD-OP - specifică codul operaţiei instrucţiunii curente

D - specifică modul de specificare a operanzilor destinaţie pentru:

D=0 R/M ← R/M [operaţie] REG;

D=1 REG ← REG [operaţie] R/M

W - stabileşte modul de operare pe octet sau cuvânt

W = 0 specifică prelucrare pe octet

W = 1 specifică prelucrare pe cuvânt

MOD - stabileşte modul în care se interpretează câmpul R/M care precizează fie codul unui registru, fie modul de calcul al adresei unei locaţii de memorie(tabelul 6.1)

REG - specifică adresa unuia din operanzi care este în REG sau o extensie a codului operaţiei(tabelul 6.2).

R/M - specifică registru ce conţine un operand sau registru utilizat în calculul adresei efective în funcţie de câmpul MOD (tabelul 6.3).

Tabelul 6.1. Codificarea câmpului MOD

MOD = 00	Un operand este în memorie, adresa efectivă nu utilizează deplasament
MOD = 01	Un operand este în memorie, adresa efectivă utilizează deplasament pe 8 biţi
MOD = 10	Un operand este în memorie, adresa efectivă utilizează deplasament pe 16 biţi
MOD = 11	Ambii operanzi sunt registrele generale

Tabelul 6.2. Codificarea câmpului REG

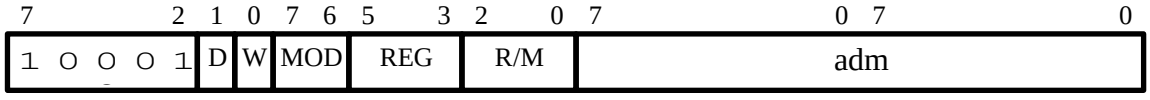
COD	REG
000	RA
001	RB
010	RC
011	IS
100	XA
101	XB
110	BA
111	BB

Tabelul 6.3. Codificarea câmpului R/M

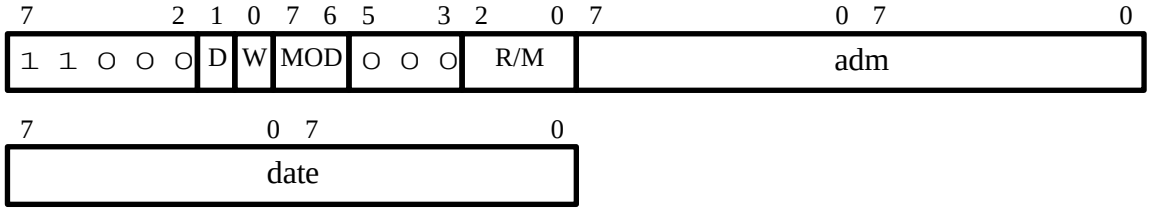
		MOD			
		00	01	10	11
R/M	000	(BX)+(SI)	(BX)+(SI)+D8	(BX)+(SI)+D16	RA
	001	(BX)+(DI)	(BX)+(DI)+D8	(BX)+(DI)+D16	RB
	010	(BP)+(SI)	(BP)+(SI)+D8	(BP)+(SI)+D16	RC
	011	(BP)+(DI)	(BP)+(DI)+D8	(BP)+(DI)+D16	IS
	100	(XA)	(SI)+D8	(SI)+D16	XA
	101	(XB)	(DI)+D8	(DI)+D16	XB
	110	(BA)	(BP)+D8	(BP)+D16	BX
	111	(BB)	(BX)+D8	(BX)+D16	BB

Exemple de format pentru codificarea instrucţiunii MOV

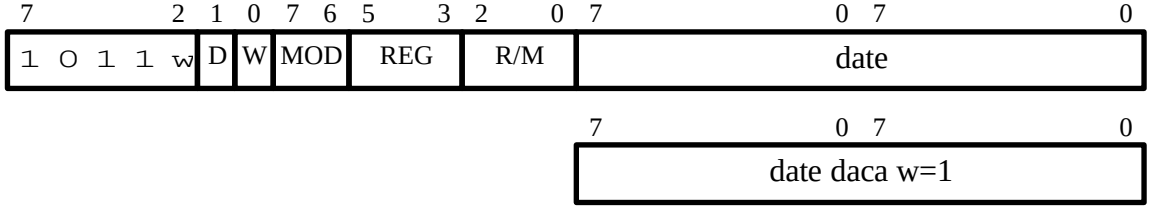
- copiere conţinut între două registre sau între un registru şi memorie:



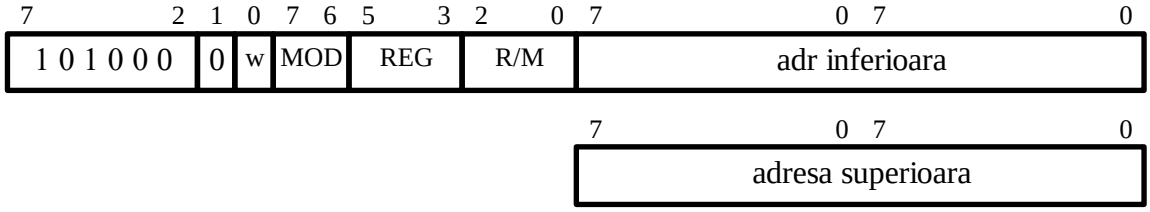
- memorare valoare în registru sau în memorie :



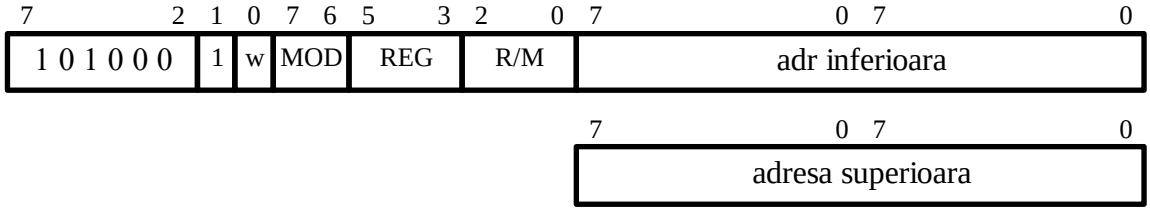
-memorare valoare în registru :



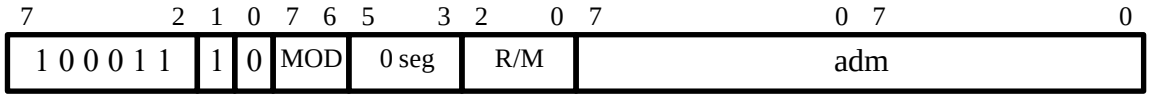
-copiere valoare din memorie în acumulator (AX sau AL) :



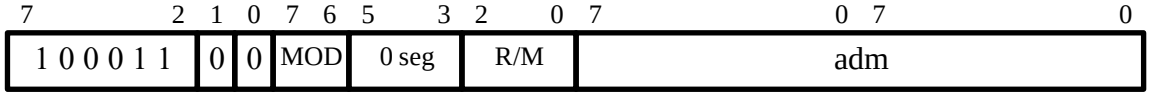
copiere valoare din acumulator (AX sau AL) în memorie :



- copiere valoare din memorie sau registru în registru segment :



copiere valoare din registru segment în memorie sau registru :



unde câmpurile din formatul instrucţiunii au următoarele semnificaţii:

dacă d = 1 atunci direcţia este spre registre altfel din registre
dacă w = 1 atunci adresare pe cuvânt (16 biţi) altfel pe octet
adm = adresă sau deplasament dacă instrucţiunea este cu referire la memorie altfel câmpul respecti
lipseşte

câmpul notat cu **mod** codifică următoarele cazuri :

mod = 11 atunci cîmpul r/m specifică un registru de 16 sau 8 biţi
mod = 00 şi r/m ≠ 110 specifică o deplasare 0
mod = 01 atunci deplasarea este de 8 biţi interpretată ca o deplasare de 16 biţi prin extinderea semnul

mod = 10 atunci deplasarea este de 16 biţi

câmpul **r/m** codifică următoarele cazuri :

r/m = 000 adresa relativă = (BX) + (SI) + deplasare
r/m = 001 adresa relativă = (BX) + (DI) + deplasare
r/m = 010 adresa relativă = (BP) + (SI) + deplasare
r/m = 011 adresa relativă = (BP) + (DI) + deplasare
r/m = 100 adresa relativă = (SI) + deplasare
r/m = 101 adresa relativă = (DI) + deplasare
r/m = 110 şi mod ≠ 00 atunci adresa relativă = (BP) + deplasare
r/m = 111 adresa relativÍ = (BX) + deplasare

dacă **mod** = 00 şi **r/m** = 110 atunci adresa relativă = deplasare

câmpul **reg** codificĺ urmĺtoarele cazuri :

w = 1		w = 0	
000	AX	000	AL
001	CX	001	CL
010	DX	010	DL
011	BX	011	BL
100	SP	100	AH
101	BP	101	CH
110	SI	110	DH
111	DI	111	BH

câmpul **seg** codificĺ urmĺtoarele cazuri :

00	ES
01	CS
10	SS
11	DS

Să considerăm câteva exemple :

adresare imediată: w reg date

```
MOV   AL,05        ; 1011 0 000  0000 0101
MOV   AX,0105H     ; 1011 1 000  0000 0101  0000 0001
```

adresare directă la registre: dw mod reg reg

```
MOV   AL,BL        ; 100010 1 0  11   000  011
MOV   AX,BX        ; 100010 1 1  11   000  011

                     mod   seg  r/m

MOV   AX,DS        ; 10001100    11   0 11   000
MOV   DS,AX        ; 10001110    11   0 11   000
```

adresare indexată: d w mod reg r/m adm

```
MOV   AL,[BX]              ; 100010 1 0  00   000  111
MOV   AX,[BX]              ; 100010 1 1  00   000  111
MOV   AL,[BX+100H]         ; 100010 1 0  10   000  111 0000 0000 0000 0001
MOV   AX,[BX+100H]         ; 100010 1 1  10   000  111 0000 0000 0000 0001
MOV   AL,[SI]              ; 100010 1 0  00   000  100
MOV   AX,[SI]              ; 100010 1 1  00   000  100
```

adresare directă: w adresa

```
MOV   AL,[100H]    ; 1010000  0  0000 0000  0000 0001
```

```
MOV   AX,[100H]     ; 1010000  1  0000 0000  0000 0001
MOV   [100H],AL     ; 1010001  0  0000 0000  0000 0001
MOV   [100H],AX     ; 1010001  1  0000 0000  0000 0001
```

Structura foarte complexă a formatului instrucţiunii face ca o aceeaşi instrucţiune să poată fi reprezentată cu ajutorul mai multor coduri de instrucţiuni. De exemplu instrucţiunea :

```
MOV     AX,102H     ; AX ←102H
```

poate să fie reprezentată cu ajutorul codului: (w mod r/m date)

110001111100000000000001000000001 (0C7H,0C0H,02H,01H)

sau a codului: (w reg date)

1011 1 000 0000 0010 0000 0001 (0B8H,02H,01H)

De asemenea instrucţiunea :

```
MOV     AX,[102H]   ; AX ← M[102H]
```

poate fi reprezentata cu ajutorul codurilor: (w adresa)

1010000 1 0000 0010 0000 0001 (0A1H,02H,01H)

sau a codului (w d mod reg r/m adm)

100010 1 1 00 000 110 0000 0010 0000 0001 (8BH,6,2,1)

6.5 Moduri de adresare

În general, operanzii cu care lucrează instrucţiunile maşinii de bază pot fi numiţi operand destinaţie şi operand sursă, rezultatul operaţiei dintre aceştia depunîndu-se în destinaţie.

Cele mai multe instrucţiuni cu doi operanzi specifică:

- operandul destinaţie, într-un registru sau într-o celulă de memorie de la adresa efectivă;
- operandul sursă, într-o celulă de memorie sau într-un registru sau constantă specificată în instrucţiune.

Operaţiile de tip memorie-memorie sunt excluse (cu excepţia celor pe şiruri de octeţi).

Instrucţiunile cu un singur operand, pot lucra cu operandul în registru sau într-o celulă de memorie. În acest caz este exclusă situaţia utilizării unui operand imediat.

Modurile de adresare ale calculatorului didactic au fost prezentate în capitolul 5.

6.6 Elementele limbajului de asamblare

Ca orice limbaj, elementele constitutive sunt:

- alfabet;
- cuvinte;
- propoziţii.

6.6.1 Alfabetul limbajului de asamblare

Majoritatea limbajelor de asamblare admit un alfabet format din 3 tipuri de caractere elementare:

- caractere alfabetice: a, b, c, ..., z, A, B, C, ..., Z
- caractere numerice: 0, 1, 2, ..., 9
- caractere speciale: ? @ _ $: . [ ] ( ) { } + - * / & % ! '# ^ | ~ \ ` "

6.6.2 Cuvinte

Cuvintele sunt şiruri de caractere definite conform anumitor reguli şi care au o anumită valoare semantică. Spre deosebire de alfabet, care este în general acelaşi pentru orice limbaj, setul de cuvinte este, în cea mai mare parte, specific limbajului.

Cuvintele se împart în două categorii:

- nume simbolice, sau identificatori;
- constante.

6.6.2.1 Identificatori

Un identificator este un şir de caractere elementare, care poate cuprinde şi caracterele speciale _ @ $?, de lungime maxim 31. Spaţiul nu este permis în cadrul simbolurilor, iar unele caractere speciale (?,$) au o anumită semnificaţie în funcţie de context.

Punctul (.) poate fi utilizat ca prim caracter pentru specificarea unor comenzi.

S-a răspîndit convenţia ca identificatorul să înceapă cu o literă.

În raport cu funcţiile pentru care se definesc, identificatorii se grupează în:

- identificatori standard, sau cuvinte cheie (cuvinte rezervate) ale limbajului;
- identificatori definiţi de utilizator, care trebuie să fie diferiţi de cuvintele cheie.

Cuvintele cheie sunt folosite pentru definirea unor funcţii şi argumente. Ele sunt proprii limbajului de asamblare şi se stabilesc la definirea acestuia, alcătuind vocabularul fix al limbajului.

Numele simbolice definite de utilizator reprezintă nume de variabile, adrese de operanzi şi instrucţiuni, nume de proceduri, segmente, macro, nume de constante.

6.6.2.2 Constantele

Constantele pot fi utilizate pentru a specifica numere sau şiruri, care sunt stabilite sau iniţializate în timpul procesului de asamblare.

Există patru tipuri de constante:

- întregi;
- reale;
- zecimale codificate binar;
- şiruri.

6.6.2.2.1 Constantele întregi

Constantele întregi reprezintă valori întregi ce pot fi utilizate în diverse contexte cum ar fi în declararea de date ca operand imediat, etc.

Constantele întregi pot fi specificate în:

binar: 101010B
octal: 32Q
zecimal: 26D
hexazecimal: 1AH

În cazul constantelor hexazecimale, trebuie ca primul caracter să fie o cifră între 0 si 9, pentru a nu se confunda cu un identificator.

De exemplu: FH este considerat identificatorul FH;

iar

0FH este constanta 15 exprimată în hexazecimal.

Dacă nu se specifică nici un tip de reprezentare (B, Q, O, D, H ca terminator de constantă), se consideră implicit reprezentarea zecimală.

Exista o comandă care permite modificarea acestei convenţii:

.RADIX expresie

unde expresie, în urma evaluării, trebuie să fie 2, 8, 10, 16.

O constantă întreagă poate fi reprezentată pe 8, 16, 32, sau 64 de biţi.

6.6.2.2.2 Constante reale

Constantele reale sunt exprimate prin mantisă şi exponent şi sunt reprezentate uzual în format zecimal.

Sintaxa unei constante reale este:

[±] întreg.fracţie [E ± exponent]

Numărul de cifre pentru reprezentare poate fi 8, 16, sau 20, ceea ce este echivalent cu 32, 64 sau 80 biţi. Constantele reale pot să apară numai în inţializarea datelor.

Exemplu:

```
DD  25.23          reprezentare pe 32 biţi
DQ  2.523 E1             reprezentare pe 64 biţi
DT  2523.0 E-2           reprezentare pe 80 biţi
```

În reprezentarea Microsoft, utilizată de MASM, constanta 1.0 se reprezintă

```
DD  81 00 00 00 h
DQ  81 00 00 00 00 00 00 00 h,
```

iar în reprezentarea IEEE, utilizată de TASM constanta 1.0 se reprezintă:

```
DD  3F 80 00 00 h
DQ  3F F0 00 00 00 00 00 00 h,
```

iar ca operand real temporar pe 80 de biţi se reprezintă:

```
DT  3F FF 80 00 00 00 00 00 00 00 h.
```

6.6.2.2.3 Constante zecimal codificate binar

Constantele întregi pot fi codificate şi în BCD (zecimal codificat binar), în care fiecare cifră zecimală are o reprezentare zecimală pe 4 biţi. Aceste constante pot fi reprezentate numai pe 80 de biţi. Reprezentarea BCD poate să apară numai în initializarea datelor

Exemplu:

```
DT  1234567890 va fi codificată: 00 00 00 00 00 12 34 56 78 90 h
DT -1234567890 va fi codificată: 80 00 00 00 00 12 34 56 78 90 h
```

6.6.2.2.4 Constante exprimate prin şiruri

O constantă de tip şir constă într-o succesiune de caractere incluse între apostrof(') sau ghilimele(").

Sintaxa:

```
'şir de caractere'
"şir de caractere".
```

Exemplu:

```
'a'
"a"
"Acesta este un mesaj"
"Această""valoare""este o constantă"
```

'Această "valoare" este o constantă'
Dacă în şirul de caractere trebuie să apară ' sau ", acestea trebuie dublate.

6.6.3 Propoziţii

În limbaj de asamblare, o propoziţie poate avea maximum 128 de caractere şi reprezintă o instrucţiune maşină sau o pseudoinstrucţiune. Pseudoinstrucţiunile nu au ca echivalent instrucţiuni maşină, avînd rolul de a transmite asamblorului indicaţii referitoare la modul său de lucru.

Peudoinstrucţiunile au ca scop principal:

- definirea de constante;
- rezervarea de zone de memorie;
- controlul asamblării;
- etc.

Sintaxa generală pentru o instrucţiune în limbaj de asamblare este următoarea:

[<eticheta>:][<mnemonică_operaţie>[<operanzi>[<comentarii>]]

<etichetă> - este un nume simbolic format din maximum 31 de caractere alfanumerice şi caracterele speciale - ? @ $ din care primul caracter este o literă sau un caracter special. Orice etichetă are asociată o valoare, şi anume adresa relativă, faţă de segmentul sau grupul din care face parte, a primului octet din instrucţiunea etichetată;

<mnemonică_operaţie> - este mnemonica corespunzătoare instrucţiunii.

<operanzi> - este un câmp ale cărui existenţă şi formă depind de tipul instrucţiunii. Acest câmp este exprimat prin expresii care pot cuprinde constante, nume de regiştri, identificatori utilizând operatori aritmetici, logici, operaţionali, de conversie, de deplasare, de tip, etc.

<comentariu> -este un câmp care începe cu ";" şi conţine un text ce va fi neglijat de asamblor. Are rolul de a preciza unele informaţii programatorului.

6.6.3.1 Operatori

De menţionat că evaluarea expresiilor se face la momentul asamblării, deci valorile utilizate ca operanzi trebuie să permită acest lucru.

Operatorii utilizaţi în expresii pot fi grupaţi în următoarele categorii:

6.6.3.1.1 Operatori aritmetici

Sunt utilizaţi în operaţii matematice şi sunt prezentaţi în tabelul următor:

OPERATOR	SINTAXA	
+	+expresie	operator unar, pozitiv
-	- expresie	operator unar negativ
+	expr1 + expr2	adunare
-	expr1 - expr2	scădere
*	expr1 * expr2	înmulţire
/	expr1 / expr2	împărţire

MOD	expr1 MOD expr2	modulo

6.6.3.1.2 Operatori de deplasare

Operatorii de deplasare pot fi utilizaţi pentru calculul valorilor oţinute prin deplasarea unei constante cu un număr de biţi la dreapta sau la stânga.

Sintaxă:

```
expresie SHR nr     ;deplasează valoarea expresiei cu un ;număr de biţi la dreapta
expresie SHL nr     ;deplasează valoarea expresiei cu un ;număr de biţi la stânga.
```

Operatorii de deplasare nu trebuie confundaţi cu instrucţiunile SHR, SHL, care lucrează asupra unui registru sau asupra unei celule de memorie în timpul execuţiei, pe când operatorii SHR, SHL lucrează asupra unor constante întregi, numai în procesul asamblării.

```
01110111B SHL 3    ; 01110111000B
01110111B SHR 3    ; 01110B.
```

6.6.3.1.3 Operatori logici

Operatorii logici realizează operaţiile logice între expresii care sunt exprimate prin constante.

Sintaxa:

```
NOT  expresie                ; negare logică
expresie1  AND  expresie2    ; SI logic
expresie1  OR   expresie2    ; SAU logic
expresie1  XOR  expresie2    ; SAU exclusiv
```

Nu trebuie confundaţi cu instrucţiunile NOT, AND, OR, XOR. Instrucţiunile implică registrele şi celulele de memorie în timpul execuţiei.

Operatorii lucrează asupra unor constante întregi.

6.6.3.1.4 Operatori relaţionali

Operatorii relaţionali compară două expresii şi generează valoarea logică "true" sau valoarea logică "false" în funcţie de relaţia între cei doi operanzi. Valorile logice sunt reprezentate astfel: 0 pentru false şi -1 pentru true.

Sintaxa:

expr1 EQ expr2	true, dacă expr1 = expr2
expr1 NE expr2	true, dacă expr1 ≠ expr2
expr1 LT expr2	true, dacă expr1 < expr2
expr1 LE expr2	true, dacă expr1 Ł expr2
expr1 GT expr2	true, dacă expr1 > expr2
expr1 GE expr2	true, dacă expr1 ł expr2

6.6.3.1.5 Operatorul DUP

Acest operator este utilizat în cadrul declaraţiilor de date pentru a specifica multiplicarea de un număr de ori a unor date.

Sintaxă:

```
număr DUP (valoare_iniţială, valoare_iniţială,...)
```

unde "număr" specifică de câte ori se specifică valorile iniţiale.

Valorile iniţiale pot fi expresii ce se evaluează ca valori întregi, caractere, sau pot fi chiar alţi operatori DUP.

Se poate utiliza şi simbolul "?" dacă nu contează valoarea cu care se iniţializează.

Exemplu:

```
buffer      DB    256   DUP (?)                  ;se rezervă o zonă de 256 octeţi
mask        DB    20    DUP (40H, 20H, 04H, 02H) ;se rezervă o zonă de 80 de octeţi, grupaţi în
                                                   20 de subzone, fiecare având valorile
                                                   40,20,04,02.
vector      DB    10    DUP (1)                  ;se rezervă o zonă de 10 octeţi iniţializaţi cu
                                                   valoarea 1.
variabila   DD    5     DUP (5 DUP(5 DUP(0)))    ;se rezervă 125 de cuvinte duble, iniţializate
                                                   cu 0.
```

6.6.3.2 Prioritatea operatorilor

Expresiile se evaluează având în vedere următoarele reguli:

- Se evaluează parantezele;
- Se aplică mai întii operatorii cu prioritate maximă;
- La operatori egali, evaluarea se face de la stânga la dreapta;

Ordinea de priorităţi a operatorilor este:

Prioritate	Operator
1	+, -, (operatori unari)
2	*, / ,MOD, SHL, SHR
3	+, - (operatori binari)
4	EQ, NE, LE, LT, GT, GE
5	NOT
6	AND
7	OR, XOR

6.7 Declararea datelor în limbajul de asamblare

Declararea datelor se realizează cu ajutorul unor pseudoinstrucţiuni. Acestea permit alocarea de memorie pentru date şi, în acelaşi timp, specificarea valorilor iniţiale pentru datele alocate.

Datele pot fi specificate ca numere, şiruri, sau expresii care au ca rezultat constante. Datele sunt generate în procesul de asamblare şi scrise în fişierul obiect.

Forma generală pentru o declaraţie de date:

[nume] tip {[lista expresii] | număr DUP (lista de expresii)]}

unde:

nume - este numele prin care se va referi data respectivă. Acest nume are asociat un tip şi o valoare. Tipul rezultă din tipul datei, iar valoarea este adresa la care se va găsi în memorie în timpul execuţiei programului primul octet rezervat pentru data etichetată cu numele respectiv. Numele este reprezentat printr-un identificator.

tip - este mnemonica pseudoinstrucţiunii care specifică tipul datei. Pseudoinstrucţiunile

de care dispune limbajul de asamblare sunt:

DB - defineşte date de tip octet (byte)

DW - defineşte date de tip cuvânt (2 octeţi) utilizate de obicei pentru memorarea adreselor

DD - defineşte date de tip dublu cuvînt (4 octeţi) utilizate de obicei pentru memorarea adreselor complete

DQ - defineşte date de tip patru cuvinte (8 octeţi)

DT - defineşte date de tip zece octeţi (10 octeţi)

lista de expresii - reprezintă lista unor expresii ale căror valori reprezintă valorile cu care se vor iniţializa zonele de date rezervate pentru declaraţia respectivă. Caracterul ? care poate apare în acest câmp specifică faptul că zona de date respectivă este rezervată, dar nu este iniţializată.

număr - reprezintă factorul de multiplicare (de repetare) a listei de expresii ce urmează după operatorul "DUP".

operatorul DUP - specifică repetarea listei de expresii.

Exemple:

În exemplele ce urmează vom scoate în evidenţă adresele de memorie şi conţinutul asociat variabilelor în urma unor declaraţii de date.

Adresa	Conţinut	Eticheta	Tip	Lista de expresii
0	0016	num_1	DB	16h
1	0C 00	num_2	DW	4*3
2	???? ???? ????	num_3	DQ	?
6	0001 0002 030405	num_4	DB	1, 2, 3, 4, 5
B	10 61 62 63 64	num_5	db	'abcd'
14	0A [05 [04] 07]	num_6	db	10 dup (5 dup(4),7)
50	00 61	num_7	dw	'a'
52	0014 ....R	num_8	dd	num_6
56	62 61 00 00	num_9	dd	'ab'
5A	62 61 00 00 00 00	num_10	dq	'ab'

6.8 Pseudoinstrucţiuni

Pseudoinstrucţiunile sunt operaţii care acţionează la nivelul asamblorului şi nu se generează cod obiect pentru acestea. Ele sunt utile pentru a furniza informaţii procesului de asamblare sau pentru a facilita referirile simbolice la diferite entităţi.

6.8.1 Pseudoinstrucţiunea ORG

[et:] ORG < expresie de valori definite>

Permite fixarea originii programului sau a modulului care conţine pseudoinstrucţiunea. Valorile care intră în componenţa expresiei trebuie să fie definite înainte de momentul utilizării.

6.8.2 Pseudoinstrucţiunea EQU

<simbol> EQU <expresie>

Atribuie simbolului valoarea rezultată prin evaluarea expresiei; Simbol trebuie să fie definit în mod unic, altfel se va genera eroare de multiplă definire.

6.8.3 Pseudoinstrucţiunea =

<simbol> = <expresie>

Atribuie simbolului valoarea rezultată din evaluarea expresiei, permiţând redefinirea.

6.8.4 Pseudoinstrucţiunea DW

[nume] DW <lista de expresii>

Generează începând cu adresa asociată numelui valorile rezultate din evaluarea listei de expresii, reprezentate pe cuvinte.

6.8.5 Pseudoinstrucţiunea DD

[nume] DD <lista de expresii>

Generează începând cu adresa asociată numelui valorile rezultate din evaluarea listei de expresii, reprezentate pe dublu cuvânt.

6.8.6 Pseudoinstrucţiunea DQ

[nume] DQ <lista de expresii>

Generează începând cu adresa asociată numelui valorile rezultate din evaluarea listei de expresii, reprezentate pe patru cuvinte.